Uplisting API [Public]

The Uplisting API

Responses

401 Unauthorised

If the API key is not recognised in any API calls, the response will be 401 Unauthorised

200 OK

For GET endpoints: If the request body is formatted correctly, the response will be a 200 OK.

201 Created

For most POST endpoints, the response will be 201 with the ID of the entity created

202 Accepted

For the calendar update endpoint: If the request body is formatted correctly, the response will be a 202 Accepted. Along with a request ID as a JSON response.

Rate limiting

The API is designed to provide a list of bookings/availability as a "snapshot" and then webhooks should be used to get up-to-date information on changes in the system.

All calls to the Uplisting API are subject to rate limiting. If a partner exceeds the rate limits, a 429 Too Many Requests response is returned. We recommend to stagger requests througout a longer time frame to avoid hitting 429s.

We allow up to 5 requests per second per IP address and no more than 100 requests per minute per IP address and no more than 15 requests per min per property.

Webhook notifications

POST Webhook - Registering Hooks

https://connect.uplisting.io/hooks

To receive a push notification when a booking is created/updated/removed, or a property is created/updated/removed, you need to register a hook that we will the POST with a payload when one of the above events happen.

Booking related hooks

1. `booking_created`
   Triggers for each new booking imported from a booking site, or created on Uplisting.
2. `booking_updated`
   Triggers when changes are made to any booking.
3. `booking_removed`
   Triggers when a booking is cancelled or removed.

Example payload

When a booking is created, we will POST to that `target_url` with a payload of:

json


```
{
  id: 1,
  accommodation_management_fee: 20.0,
  accomodation_total: 362.45,
  arrival_time: "15:00:00",
  automated_messages_enabled: true,
  automated_reviews_enabled: false,
  booked_at: "2021-11-26T11:19:59Z",
  channel: "airbnb_official",
  check_in: "2021-12-03",
  check_out: "2021-12-05",
```


Other booking-related hooks you can register are for `booking_updated` and `booking_removed`. Follow the same approach as above to register a target URL for each event.

For `booking_updated`, the payload will be as above.

For `booking_removed` when a booking is cancelled the payload would also be as above with an additional attribute of `reason`.

For bookings that are deleted from the Uplisting calendar the payload is a more reduced one, e.g:

```json
{
  id: 163760,
  arrival_time: "15:00:00",
  automated_messages_enabled: true,
  automated_reviews_enabled: false,
  booked_at: "2021-11-26T12:51:01+00:00",
  channel: "uplisting",
  check_in: "2021-12-02",
  check_out: "2021-12-03",
  currency: "USD",
  departure_time: "11:00:00",
```

Booking field description

Field

channel

Description

`"airbnb"`, `"google"`, `"booking_dot_com"`, `"uplisting"`, `"homeaway"`

Field

reason

Description

Reason for the booking modification:

- `"cancelled"`
- `"checked_in"`
- `"checked_out"`
- `"confirmed"`
- `"externally_removed"`
- `"needs_check_in"`
- `"needs_check_out"`
- `"remove_pending"`
- `"removed_as_unpaid"`

Property related hooks

1. `property_created`
2. `property_updated`
3. `property_removed`

The payload for property related webhooks is:

```json
{
  id: property.id,
  name: "Mi Casa",
  nickname: "Su Casa",
  timestamp: "2021-11-26T11:20:00Z",
  time_zone: "Europe/London"
}
```

Property webhooks are sent when attributes on the property change - e.g. the name or nickname - as well as when associated amenities or address changes. The best approach to getting updated property information is to make an API request to `GET /properties/:id` for the property ID that is included in the payload.

Best practices

- **Webhooks order cannot be guaranteed**

In order to make sure your system is notified about changes webhooks are retried unless we receive a `200 OK` HTTP response. We use an exponential backoff logic and will retry to deliver the webhook for 21 times (in total for 8 days 8 hours post `timestamp` time).

To handle order issues we recommend to use the provided `timestamp` attribute, which represents the timestamp of the event in UTC timezone and ISO8601 format.
You should be discarding any webhooks with an earlier `timestamp` to prevent writing stale data.

For example:

- There's a booking with `guest_name` set to "Jon".
- `guest_name` is updated to "Sam" at `2022-03-24T11:03:00Z`.
- A `booking_updated` webhook `timestamp: 2022-03-24T11:03:00Z` is attempted to deliver changes, however we don't get a successful response. We will retry to deliver the webhook at a later

time.

- The same booking `guest_name` changes to "Ben" at `2022-03-24T11:05:00Z`.
- A `booking_updated` webhook with `timestamp: 2022-03-24T11:05:00Z` is atttempted to deliver changes and we get a successful response.
- The failed webhook with `timestamp: 2022-03-24T11:03:00Z` retries and successfully delivers `guest_name` changed to "Sam" payload.

If handled incorrectly the guest name will be set to "Sam" as that was the value for the `guest_name` attribute in the last webhook.
A better approach would be to log the `timestamp` of the last webhook for the booking (`2022-03-24T11:05:00Z`) and don't process webhooks with an earlier `timestamp` (`2022-03-24T11:05:00Z`).

- **A webhook can be received more than just once**

Due to the nature of HTTP requests you might receive a webhook more than once. While that's generally not an issue for `_updated` webhooks, `_created` or `_removed` webhook can fail as you would be trying to:
a) create a booking/property with the same Uplisting ID more than once
b) remove a booking/property for an Uplisting ID that is no longer present in your database.
We recommend you to handle these changes in an idempotent way. Your system shouldn't fail at processing same webhook more than once.

- **Endpoint Failure and Disablement**

If an endpoint fails to respond to 5 different events consecutively, our system will mark it as disabled. From that point forward, no further events will be posted to this endpoint until further notice. Please ensure your endpoint is reliable and responds within the expected timeframe to avoid disruption.

- **Response time**

Your service is expected to process the webhook and send a response within 5 seconds of receiving it. If your service doesn't respond within this timeframe, the webhook call will be marked as failed. Our system will then attempt to redeliver the webhook according to our retry mechanism outlined above. Please ensure your service is capable of processing incoming webhooks in a timely manner to avoid unnecessary retries and potential marking of your endpoint as disabled. We recommend building your service to acknowledge the webhook receipt immediately, and then process the payload asynchronously to avoid any timeouts.

HEADERS

Content-Type	application/json

Authorization	Basic MATwZDQ0YjEtMTRmZS00MmJhLTg1OGEtNmM4MDBmNjBlMTNi

Body raw (json)

```
json
```

```
{
  "target_url": "https://hooks.zapier.com/hooks/catch/11700/osfupzi/",
  "event": "booking_created"
}
```

DELETE Webhook - Removing hooks

```
https://connect.uplisting.io/hooks/1837
```

If you wish to remove a hook (e.g. unsubscribe from further events) then send `DELETE` to `https://connect.uplisting.io/hooks/:id` where id is the ID you got back when first registering the hook.

HEADERS

Content-Type	application/json
Authorization	Basic Mat4N2QfZTAtMzA3My00NjBmLTllMTMtNTE5NDY2NDE4Zjdm

GET Webhooks

```
https://connect.uplisting.io/hooks
```

Retrieving details of registered webhooks

To retrieve the details for all webhooks registered for a user, you should send a GET request to the endpoint above.

The payload will be a list of webhooks for the partner account in JSON API format.

For more info on JSON API format, visit https://jsonapi.org.

HEADERS

Authorization	Basic YzhlZGU3NDctNDUzNi00MGVmLWJkYWMtMzZjZjVlMGQxZDVk

v2

GET Custom booking attributes

```
https://connect.uplisting.io/v2/custom_booking_attributes
```

Listing booking attributes that can be changed

There are only certain attributes that can be changed on any specific booking and those are listed under this endpoint.

The use case here is to retrieve the list of attributes for a specific account using a host's API key. These can then be mapped to attributes on a partner's side and used with the `PATCH /v2/bookings/:id` endpoint to make changes to the booking.

These values are then used in Uplisting in automated messaging by replacing special tags used in messaging templates.

NOTE: This endpoint requires a partner client ID in the header under the `X-Uplisting-Client-ID` header. Contact Uplisting if you don't have a partner specific client ID on partner@uplisting.io.

HEADERS

Authorization	Basic ODM1ZjBjMTItNWM3Mi00ZDA2LWFkY2MtYzA3MjYwYTBjMDNh
X-Uplisting-Client-Id	8a818455-dbfb-4fc2-951b-6ff7e8b546ae

PATCH Update booking attributes

https://connect.uplisting.io/v2/bookings/123456

Any attribute listed in the custom booking attributes endpoint can be updated via this endpoint and values set against a specific booking.

Attribute values can be anything that is represented by a string, e.g.

json

```
{
    "data": {
        "attributes": {
            "lock_code": 1234567
        }
    }
}
```

These attributes can then be used by the host in automated message tags.

Values are accepted with an HTTP 201 response.

NOTE: This endpoint requires a partner client ID in the header under the `X-Uplisting-Client-ID` header. Contact Uplisting if you don't have a partner specific client ID.

HEADERS

Authorization	Basic ODM1ZjBjMTItNWM3Mi00ZDA2LWFkY2MtYzA3MjYwYTBjMDNh
X-Uplisting-Client-Id	8a818455-dbfb-4fc2-951b-6ff7e8b546ae
Content-Type	application/json

POST Create custom booking attributes

https://connect.uplisting.io/v2/custom_booking_attributes

Description

Creates a new custom booking attribute that can be used to store additional metadata for bookings.

Each attribute is tied to the partner API client that created it and must follow specific naming conventions.

Constraints

- **Maximum:** 15 custom booking attributes per partner per account.
- **Namespace Enforcement:**
  Attribute names must be namespaced (e.g., `partnername_attribute_name`).
- **Naming Convention:**
  Only underscore-style (`snake_case`) names are allowed.

⚠️ **Note:** Partner clients must have their `custom_booking_attribute_prefix` set before they can create custom booking attributes. (Please contact our Partner team on partner@uplisting.io to have that configured)

Values are accepted with an HTTP 201 response.

NOTE: This endpoint requires a partner client ID in the header under the `X-Uplisting-Client-ID` header. Contact Uplisting if you don't have a partner specific client ID.

HEADERS

Authorization	Basic ODM1ZjBjMTItNWM3Mi00ZDA2LWFkY2MtYzA3MjYwYTBjMDNh
X-Uplisting-Client-Id	8a818455-dbfb-4fc2-951b-6ff7e8b546ae
Content-Type	application/json

Body raw (json)

json

```
{
    "data": {
        "attributes": {
            "name": "partner_custom_attribute_name",
            "description": "Describe the custom attribute here"
        }
    }
}
```

POST Create booking

https://connect.uplisting.io/v2/bookings

Endpoint allows creating confirmed bookings and accepts the following attributes:

- `check_in` - required, date in ISO8601 format.
- `check_out` - required, date in ISO8601 format, date must be after `check_in` date.
- `property ID` - required.

Optional attributes:

- `guest_name` - optional, guest name shown on the booking calendar and used for automated messaging.
- `guest_email` - optional, a valid email is required for any scheduled messages to be delivered to the guest.
- `guest_phone` - optional.
- `number_of_guests` - optional, used for price calculation.

NOTE: This endpoint requires a partner client ID in the header under the `X-Uplisting-Client-ID` header. Contact Uplisting if you don't have a partner specific client ID.

HEADERS

Authorization	Basic ODM1ZjBjMTItNWM3Mi00ZDA2LWFkY2MtYzA3MjYwYTBjMDNh
X-Uplisting-Client-Id	8a818455-dbfb-4fc2-951b-6ff7e8b546ae
Content-Type	application/json

Body raw (json)

json

```
{
    "data": {
        "attributes": {
            "check_in": "2022-06-30",
            "check_out": "2022-07-06",
            "guest_name": "Jon Snow",
            "guest_email": "jon@example.com",
            "guest_phone": "+001122334455",
            "number_of_guests": 3
        },
        "relationships": {
```

GET Verify API key

https://connect.uplisting.io/users/me

Initially, you need to verify that your API key is correct and that you are using the correct authentication header.
To do so, `GET` the following URL: https://connect.uplisting.io/users/me with an authorisation header that is your **API key encoded with Base64** with application/json content-type.

For example, in Ruby, these are the headers you need to pass.

Plain Text

```
Hash["HTTP_AUTHORIZATION" => "Basic #{Base64.encode64(api_key)}", "CONTENT_TYPE" => "
```

If successful, you should get back the name of your account and a unique ID for the user.

Plain Text


```
{
    "name": "Uplisting",
    "uid": "7005ddf20990ceaac970415da5eb86cd29c4c7323f527c66f18978f8e36edd7a"
}
```


This proves that you have the correct authorisation header, which should be used for all future communication.

If you receive an error that states "Your API key does not appear to be valid" please first check you have Base64 encoded the key. Otherwise please reach out to partner@uplisting.io for further assistance including the email address of the Account owner.

HEADERS

Authorization	Basic NDA3Y2U5NTktOTI1MS00ZWM2LTkxMjQtZjFjOWFmZDBjYWYy
Content-Type	application/json

GET Properties

https://connect.uplisting.io/properties

Retrieving details for all properties

To retrieve the details for all properties on an Uplisting account, you should send a GET request to the endpoint above.

The payload will be a list of properties for the partner account in JSON API format. It will also include the

property address, photos, multi-units, fees, taxes, discounts, suitabilities and amenities in the included data.

For a full example of the JSON response, see the property endpoint. This endpoint will return an array of properties with the same format as a single property

For more info on JSON API format, visit https://jsonapi.org.

Relationships

Properties have relationships which include address, photos, multi-units, fees, taxes, discounts, suitabilities, policies, channel commissions and amenities types. Each individual relationship references the relevant resource in the included section of the API response, per the JSON API format.

Addresses

Our example property's address relationship is provided as follows:

Plain Text

```
"address": {
  "data": {
    "id": "15111",
    "type": "addresses"
  }
}
```

And the included data has the relevant address.

Plain Text

```
{
  "id": "15111",
  "type": "addresses",
  "attributes": {
   "street": "3503 B Street",
   "suite": null,
   "city": "Philadelphia",
   "state": "PA",
   "zip_code": "19134",
   "country": "United States",
   "latitude": 40.003384,
```

Channel commissions

Our example of channel commissions relationship is provided as follows:

Plain Text

```
"channel_commissions": {
  "data": [
    {
        "id": "7775",
      "type": "channel_commissions"
    },
    {
      "id": "7776",
      "type": "channel_commissions"
    },
    {
```

And the included data has the relevant channel commissions.

Plain Text

```
[
  {
    "id": "7775",
    "type": "channel_commissions",
    "attributes": {
      "channel": "airbnb_official",
      "amount": 18.34
    }
  },
  {
    "id": "7776",
```

Photos

For example our a property's photo relationship is provided as follows:

Plain Text

```
"photos": {
   "data": [
     {
       "id": "159361",
       "type": "photos"
     }
   ]
}
```

And the included data has the relevant photo.

Plain Text


```
{
  "id": "159361",
  "type": "photos",
  "attributes": {
    "url": "https://cdn.filestackcontent.com/yEswEYOzRaiOBsk90s3w",
    "order": 1,
    "created_at": "2020-01-16T21:55:50Z",
    "updated_at": "2020-01-16T21:55:50Z"
  }
},
```


Multi units

If a property has associated multi-units, these will be included in the JSON API response.

For example:

Plain Text


```
"multi_units": {
  "data": [
    {
      "id": "68",
      "type": "multi_units"
    },
    {
      "id": "69",
      "type": "multi_units"
    }
  ]
```

And the included data has the relevant multi_unit:

```
{
  "id": "68",
  "type": "multi_units",
  "attributes": {
    "name": "5G"
  }
}
```

Taxes

All taxes for a property are included, even if those taxes are zero.

The taxes have a `type` of `fixed` for a specific amount and `percentage` for percentages.

They also have a field called `per` which indicates whether the tax applies per `booking` (e.g. for the whole stay), per `night` (e.g. for each night of the stay) or per `person_per_night` (e.g. per person for each night of the stay).

The `amount` field indicates the fixed amount, or the percentage amount to apply.

For example:

Plain Text


```
"taxes": {
  "data": [
    {
      "id": "11033-per_booking_amount",
      "type": "property_taxes"
    },
    {
      "id": "11033-per_booking_percentage",
      "type": "property_taxes"
    },
    {
```


And the included taxes have the relevant data:

Plain Text


```
{
  "id": "11033-per_booking_percentage",
  "type": "property_taxes",
  "attributes": {
    "name": "Per booking percentage",
    "label": "per_booking_percentage",
    "type": "percentage",
    "per": "booking",
    "amount": 25.25
  }
}
```


Fees

All fees - e.g. cleaning fee and/or extra guest charge - for a property are included, if they are setup

All fees - e.g. cleaning fee and/or extra guest charge - for a property are included, if they are setup.

The fees have a field of `enabled` which indicates if they should be applied.

They also have a field called `amount` that indicates the actual fee that should be applied.

There's a field named `guests_included` that indicates, for the extra guest charge fee, how many guests should be included *before* the fee applies. For example, guests_included 2 means that the guest fee only applies for guests 3 and 4 (for a booking with 4 guests).

For example:

Plain Text

```
"fees": {
  "data": [
    {
      "id": "11033-cleaning_fee",
      "type": "property_fees"
    },
    {
      "id": "11033-extra_guest_charge",
      "type": "property_fees"
    }
  ]
```

And the included fees have the relevant data:

Plain Text

```
{
  "id": "11033-cleaning_fee",
  "type": "property_fees",
  "attributes": {
    "name": "Cleaning fee",
    "label": "cleaning_fee",
    "enabled": true,
    "guests_included": null,
    "amount": 100.0
  }
},
```

Discounts

All discounts for a property are included, even if those discounts are zero

All discounts for a property are included, even if those discounts are zero.

The discounts have a `type` of `percentage` for percentages. (The only type currently supported).

They also have a field called `days` which indicates how many days the discount applies for (e.g. 7 for a week long stay or 28 for a monthly long stay)

The `amount` field indicates the percentage amount to apply.

For example:

Plain Text

```
"discounts": {
  "data": [
    {
      "id": "11033-weekly",
      "type": "property_discounts"
    },
    {
      "id": "11033-monthly",
      "type": "property_discounts"
    }
  ]
```

And the included discounts have the relevant data:

Plain Text

```
{
  "id": "11033-weekly",
  "type": "property_discounts",
  "attributes": {
    "name": "Weekly discount",
    "label": "weekly",
    "type": "percentage",
    "days": 7,
    "amount": 5.0
  }
}
```

Suitabilities

This section indicates whether a property is suitable for `pets` `children` `events` or `smoking`

This section indicates whether a property is suitable for `pets`, `children`, `events` or `smoking`.

Each property will have this section:

```
"suitability": {
  "data": {
    "id": "3428",
    "type": "suitabilities"
  }
}
```

And the included data:

```
{
  "id": "3428",
  "type": "suitabilities",
  "attributes": {
    "children": true,
    "pets": false,
    "events": false,
    "smoking": false
  }
}
```

Policy

This section indicates what cancellation policy a property has.

Each property will have this section:

```
```

```
"policy": {
  "data": {
    "id": "1234",
    "type": "policies"
  }
}
```

And the included data:

Plain Text

```
{
  "id": "3428",
  "type": "policies",
  "attributes": {
    "type": "Strict cancellation policy",
    "description": "Cancel any time before check-in and no refund is provided"
  }
}
```

Protect Security Deposit:

This section indicates whether the Uplisting Protect (security deposit) feature is turned on for the property.

Each property will have this section if Uplisting Protect has been enabled for the account.

Plain Text

```
"protect_security_deposit_setting": {
  "data": {
    "id": "1234",
    "type": "protect_security_deposit_settings"
  }
}
```

And the included data:

Plain Text

```
{
```

```
}
  "id": "3428",
  "type": "protect_security_deposit_settings",
  "attributes": {
    "amount": 200.0,
    "enabled": true
  }
}
```

Amenities

The Properties Index endpoint contains amenities for each property. The full list of amenities Uplisting supports are listed below.

In our example the amenity relationship is provided as:

Plain Text

```
"amenities": {
   "data": [
      {
        "id": "1",
        "type": "amenities"
      }
   ]
}
```

And the included data has the relevant amenity:

Plain Text

```
{
  "id": "1",
  "type": "amenities",
  attributes": {
    "name": "Oven",
    "group": "Kitchen & Dining"
  }
}
```

Full list of amenities supported on Uplisting

Plain Text

```
| *Amenity Name*            | *Amenity Group*       |
-------------------------------------------------------
| Satellite / Cable         | Entertainment         |
| Games                     | Entertainment         |
| Foosball                  | Entertainment         |
| DVD Player                | Entertainment         |
| Books                     | Entertainment         |
| Ping Pong Table           | Entertainment         |
| Pool Table                | Entertainment         |
| Game Room                 | Entertainment         |
| Music Library             | Entertainment         |
```

HEADERS

Content-Type	application/json
Authorization	Basic YzhlZGU3NDctNDUzNi00MGVmLWJkYWMtMzZjZjVlMGQxZDVk Base64 encoded API key

GET Property

https://connect.uplisting.io/properties/:id

Retrieving details for a single property

To retrieve the details for a property on an Uplisting account, you should send a GET request to the endpoint above.

The payload will be a single property for the partner account in JSON API format. It will also include the property's address, photos, multi-unit, fees, taxes, suitabilities, channel commissions and amenities in the included data.

For more info on JSON API format, visit https://jsonapi.org.

Relationships

Properties have relationships which include address, photos, multi-unit, fees, taxes, suitabilities and amenities types. Each individual relationship references the relevant resource in the included section of the API response, per the JSON API format.

For a full breakdown, review the documentation for the `GET /properties` endpoint

HEADERS

Content-Type	application/json
Authorization	Basic YzhlZGU3NDctNDUzNi00MGVmLWJkYWMtMzZjZjVlMGQxZDVk Base64 encoded API key

PATH VARIABLES

id

GET Availability

```
https://connect.uplisting.io/availability?min_price=50&check_in=2021-07-30&check_out=2021-08-04&number_of_guests=2&max_price=4000&city=London
```

Retrieving availability for properties

To retrieve the availability of properties on an Uplisting account, you should send a GET request to the endpoint above.

The payload will be a list of properties that meet the search criteria entered in the query string, in JSON API format. It will also include the property addresses, photos and amenities in the included data. (for a fuller explanation, see the properties endpoint)

Search criteria

You can choose to search by any, or all, of these criteria via the query string on the request.

1. `check_in` - the date of check in.

2. `check_out` - the date of check out (note this is the *morning* of check out)
3. `number_of_guests` - the number of guests for the booking
4. `max_price` - the maximum price allowed for the booking
5. `min_price` - the minimum price expected for a booking
6. `city` - scope the properties by a specific city location

HEADERS

Authorization	Basic MAIwZ44FFQ0YjEtMTRmZS00MmJhLTg1OGEtNmM4MDBmNjBlMTNi

PARAMS

min_price	50
check_in	2021-07-30
check_out	2021-08-04
number_of_guests	2
max_price	4000
city	London

GET Bookings

```
https://connect.uplisting.io/bookings/:listing_id?from=2020-12-10&to=2020-12-20
```

To retrieve the bookings for a property, you should GET the above endpoint.

The response is limited to a maximum of 50 bookings at a time but is paginated so more bookings can be retrieved, as required.

Date range options

- Specify neither `from` or `to` and get a year from today (UTC based)
  `https://connect.uplisting.io/bookings/:listing_id`
- Specify a `from` only to get 12 months worth of data from that point
  `https://connect.uplisting.io/bookings/:listing_id?from=2020-02-01`
- Specify `from` and `to` and get paginated bookings within that date range
- `https://connect.uplisting.io/bookings/:listing_id?from=2020-02-01&to=2021-12-01`

Pagination options

- Specify `page=X` to get a 0-based page of results.
- Specify `per_page=X` to retrieve less than the default of 50 in each page
- There's `meta` information included in the response that tells you `total` bookings and `total_pages` so you can paginate through as required.

Booking statuses
Bookings can have the following statuses.

- `confirmed` [all bookings are in a confirmed state until one of the below statuses takes affect]
- `checked_in` [a confirmed booking which has been marked as checked-in by the host]
- `checked_out` [a confirmed booking which has been marked as checked-out by the host]
- `needs_check_in` [a confirmed booking which is waiting to be checked-in by the host. The host may not mark this booking as checked-in]
- `needs_check_out` [a confirmed booking which is waiting to be checked-out by the host. The host may not mark this booking as checked-out]
- `cancelled` [a booking which is now cancelled, ie. the booking is not longer active]

Response

The response is the same payload as for a `booking_updated` webhook repeated for each booking retrieved. All bookings are available, including `cancelled` bookings. See the example for more info.

HEADERS

Content-Type	application/json
Authorization	Basic YzhlZGU3NDctNDUzNi00MGVmLWJkYWMtMzZjZjVlMGQxZDVk

Base64 encoded API key

PARAMS

from	2020-12-10
to	2020-12-20

PATH VARIABLES

listing_id

GET Calendar

https://connect.uplisting.io/calendar/:listing_id?from=2020-12-10&to=2020-12-20

To retrieve the calendar for a property, you should GET to the above endpoint.

The response is limited to 12 months at one time.

Date range options

- Specify neither `from` or `to` and get a year from today `https://connect.uplisting.io/calendar/:listing_id`
- Specify a `from` only to get 12 months worth of data from that point `https://connect.uplisting.io/calendar/:listing_id?from=2020-02-01`
- Specify `from` and `to` and get up up to 12 months worth of data within that date range
- `https://connect.uplisting.io/calendar/:listing_id?from=2020-02-01&to=2021-12-01`

Response

Included in the response is the price, and restrictions such as availability, minimum length of stay and closed for arrival, e.g:

Plain Text

```
"available": true,
"available_count: 1,
"date": "2020-12-10",
"day_rate": 150,
"minimum_length_of_stay": 5,
"maximum_nights_available": 15,
"closed_for_arrival": false,
"closed_for_departure": false
```

`available` : this is a simple true/false as to whether the property is available on the date listed.

`available_count` : this indicates how many properties are available on that date. It will be 1 unless a property has multi-units attached to it. For multi-units, it indicates how many units are available on that date.

`date` : the day the details are for

`day_rate` : the price (in the property's currency)

`minimum_length_of_stay` : The minimum length for any booking that starts on this date.

`maximum_nights_available` : How many nights can be booking in a contiguous block. Note that this may actually be *less* than the `minimum_length_of_stay` because the MLOS is a property-specific restriction and is not date-specific.

`closed_for_arrival` : true/false as to whether a booking can check-in on this date.

`closed_for_departure` : true/false as to whether a booking can check-out on this date.

HEADERS

Content-Type	application/json
Authorization	Basic YzhlZGU3NDctNDUzNi00MGVmLWJkYWMtMzZjZjVlMGQxZDVk Base64 encoded API key

PARAMS

from	2020-12-10
to	2020-12-20

PATH VARIABLES

listing_id

POST Update calendar

```
https://connect.uplisting.io/calendar/:listing_id
```

Uplisting provides calendar-based endpoints for updating rates, availability and minimum length of stay for a specific date range. Any updates made via the API are broadcast to all channels the property is connected to on Uplisting.

Notification

This POST results in an asynchronous response to the server listed at the notification_url with a payload indicating success/failure of the changes requested and including that request ID so the response can be matched to the initial POST.

The notification URL is not required. If no URL is provided, no notifications will be sent.

We recommend using ngrok (https://ngrok.com) to create a publicly accessible URL that tunnels back to your development server for testing purposes. They have a free tier that will enable you to test the Uplisting API.

Delay

The changes received in the payload to this POST call are not applied immediately to the property. Instead, the payload is scheduled for processing by Uplisting and will be applied to the property asynchronously. The payload contains a notification_url that is used to provide a response to the partner indicating success or failure of the relevant updates.

This delay is typically less than 1 minute.

This delay is typically less than 1 minute.

Note

- If the property ID is not part of the account specified by the API key, the notification URL will not receive a response.
- If the payload is considered invalid, the response will be `400 Bad Request` with human readable errors indicating why the payload is considered invalid.
- The date the rate, availability, MLOS update can either be a single day, specified as `date` or as a date range using `from` and `to`. Note that the night of the to date is always excluded, as it's considered the morning only. So, to include a range *including* all nights from `2020-12-01` to the night of `2020-12-10`, the to should be set to `2020-12-11`.
- All attributes can be set for up to 3 years in advance. **Any dates before "today" in UTC will be ignored**.
- Availability is specified either as `true` if the property can be booked or `false` if the property is not available to be booked. The availability specified here is for the night of the date specified, or as above to the night before the `to` date.
- The rate for the date, or date range, is specified in the currency of the property that is set on Uplisting. It should be a "commission-free" rate as appropriate markup will be applied to this rate when syncing prices to connected channels.
- `available`, `minimum_length_of_stay`, `day_rate`, `closed_for_arrival,` and `closed_for_departure` are all optional attributes, meaning you can set a rate and/or availability and/or minimum length of stay and/or closed for arrival and departure restrictions.
- Any other attributes added to the payload will be ignored.

Notification URL response

The notification URL is specified by the partner and can have any valid format. It must be an HTTPS endpoint.

A Base64 encoded partner API key will be included in the `HTTP_AUTHORIZATION` header to confirm that the response is from Uplisting.

The response will be a list of the correctly applied rate and/or availability and/or MLOS as listed below:

`{ "request_id": "fe845d4e-9fa2-4f85-b6ee-680d7f97803e" "calendar": { "days": [ { "available": true, "day_rate": 66, "date": "2019-12-20", "minimum_length_of_stay":1, "closed_for_arrival": false, "closed_for_departure": true }, { "available": false, "from": "2019-12-31", "to": "2020-01-31" } ] } }`

HEADERS

Content-Type	application/json
Authorization	Basic YzhlZGU3NDctNDUzNi00MGVmLWJkYWMtMzZjZjVlMGQxZDVk Base64 encoded API key

PATH VARIABLES

listing_id

Body raw


```
{
    "notification_url": "https://yourdomain.eu.ngrok.io/notification",
    "calendar": {
        "days": [
            {
               "available": true,
                "date": "2020-12-13",
                "day_rate": 170.0,
                "minimum_length_of_stay": 2,
                "closed_for_arrival": false,
                "closed_for_departure": true
            },
            {
```